

UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Máquina de Turing de Duas Fitas para Operações Lógicas Bit a Bit

Pedro Lucas Moreira Pinto
Pedro Ryan Oliveira de Almeida

Relatório – Teoria da Computação – 2026.2 – Outubro de 2026

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Pedro Ryan Oliveira de Almeida

Máquina de Turing de Duas Fitas para Operações Lógicas Bit a Bit

Relatório apresentado como parte das atividades da disciplina Teoria da Computação – 2026.2, Curso de Ciência da Computação, Universidade Federal do Tocantins.

PALMAS / TO

2026

SUMÁRIO

0.1 Objetivo 7

1 DESENVOLVIMENTO 8

1.1 Diagrama de estados 8

1.2 Funcionamento 8

1.2.1 Fase 1 — Seleção da operação (q_0) 8

1.2.2 Fase 2 — Cópia do primeiro operando (q_1, q_2, q_3) 8

1.2.3 Fase 3 — Retorno da fita 2 (q_4, q_7, q_9) 9

1.2.4 Fase 4 — Cálculo bit a bit (q_5, q_8, q_{10}) 9

1.3 Exemplos de execução 9

1.3.1 Rastreamento de `A110#101` 9

1.4 Limitações 10

2 CÓDIGO-FONTE 11

2.1 Módulo `fita.py`: representação das fitas 11

2.2 Importações e constantes 12

2.3 Função de transição δ 12

2.4 Descrição das fases 13

2.5 Classe `MaquinaTuring` 14

2.6 Exibição da execução 15

2.7 Ponto de entrada 16

2.8 Exemplo de saída 16

3 CONCLUSÃO 17

LISTA DE FIGURAS

Figura 1 – Diagrama de estados da Maquina de Turing no JFLAP 8

Figura 2 – Saída do simulador para a entrada `A110#101` 16

LISTA DE TABELAS

Tabela 1 – Símbolos de operação aceitos 7

Tabela 2 – Valor escrito na fita 2 em cada estado de cálculo 9

Tabela 3 – Exemplos de entradas e resultados 9

LISTA DE CÓDIGOS

2.1 Módulo `fita.py`: representação das fitas 11
2.2 Importações e constantes 12
2.3 Função de transição δ 12
2.4 Descrição das fases 13
2.5 Classe `MaquinaTuring` 14
2.6 Exibição da execução 15
2.7 Ponto de entrada 16

0.1 Objetivo

Construir uma Maquina de Turing que, dada uma entrada na forma

$$op\ w_1 \# w_2, \qquad w_1, w_2 \in \{0,1\}^n,$$

calcule w_1 *op* w_2 bit a bit, deixando o resultado na fita 2, onde *op* é uma das operações da Tabela 1.

Tabela 1 – Símbolos de operação aceitos

Símbolo	Operação	Ramo de estados
`A`	AND $(\wedge)$	$q_1 \to q_4 \to q_5$
`X`	XOR $(\oplus)$	$q_2 \to q_7 \to q_8$
`O`	OR $(\vee)$	$q_3 \to q_9 \to q_{10}$

1 DESENVOLVIMENTO

1.1 Diagrama de estados

A Figura 1 mostra o diagrama construído no JFLAP. A máquina possui três ramos paralelos e simétricos, um para cada operação, que convergem no estado de aceitação q_6.

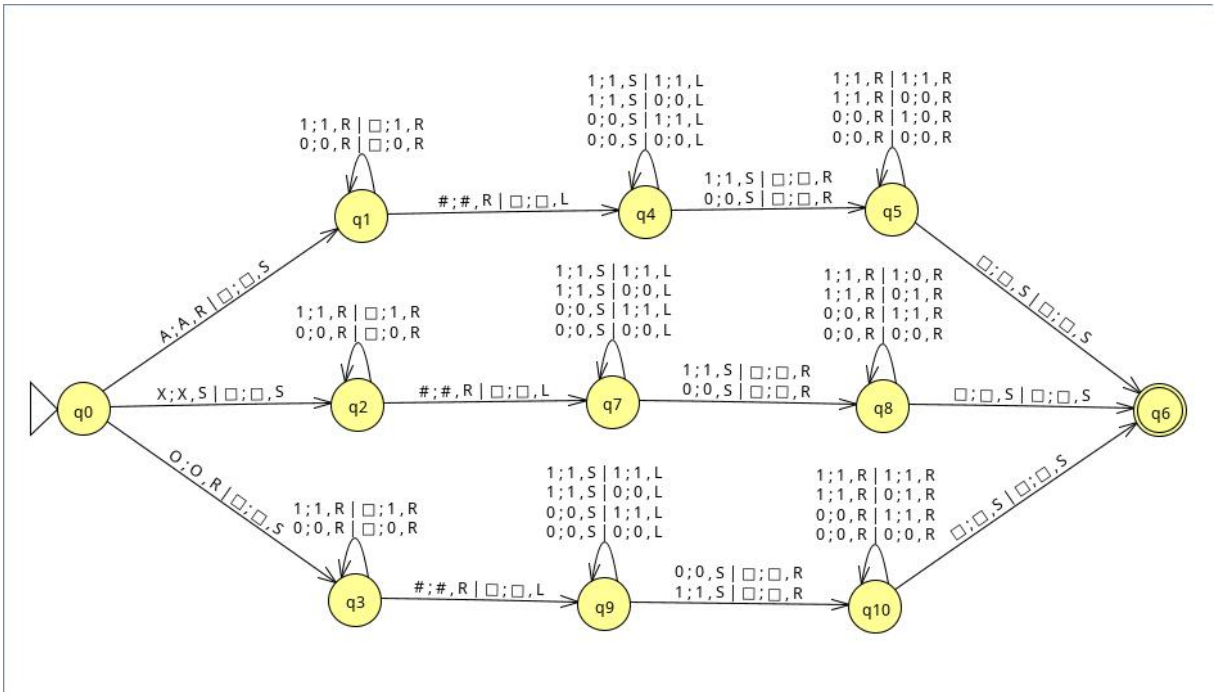


Figura 1 – Diagrama de estados da Maquina de Turing no JFLAP

1.2 Funcionamento

Inicialmente a fita 1 contém a entrada e a fita 2 está vazia. O processamento ocorre em quatro fases.

1.2.1 Fase 1 — Seleção da operação (q_0)

Em q_0 a máquina lê o primeiro símbolo da fita 1, que identifica a operação, avança a cabeça da fita 1 e desvia para o ramo correspondente: `A` $\rightarrow q_1$, `X` $\rightarrow q_2$, `0` $\rightarrow q_3$.

1.2.2 Fase 2 — Cópia do primeiro operando (q_1, q_2, q_3)

Cada bit de w_1 lido na fita 1 é escrito na fita 2, e ambas as cabeças avançam para a direita (`0;0,R | □;0,R` e `1;1,R | □;1,R`). Ao encontrar `#`, a cabeça da fita 1 passa

ao primeiro bit de w_2 e a da fita 2 recua para o último bit copiado, indo para q_4, q_7 ou q_9.

1.2.3 Fase 3 — Retorno da fita 2 (q_4, q_7, q_9)

A cabeça da fita 1 permanece parada sobre o primeiro bit de w_2 enquanto a da fita 2 se desloca para a esquerda sobre os bits de w_1. Ao alcançar o branco à esquerda de w_1, ela avança uma posição e a máquina passa para q_5, q_8 ou q_{10}, com as duas cabeças alinhadas no bit mais significativo de cada operando.

1.2.4 Fase 4 — Cálculo bit a bit (q_5, q_8, q_{10})

Ambas as cabeças avançam juntas. A cada passo, o bit b_1 da fita 1 (de w_2) e o bit b_2 da fita 2 (de w_1) são combinados e o resultado sobrescreve b_2 na fita 2. A Tabela 2 resume as transições de laço de cada estado. Quando as duas fitas leem $\square$, a máquina transita para q_6 e aceita.

Tabela 2 – Valor escrito na fita 2 em cada estado de cálculo

Fita 1	**Fita 2**	q_5 (AND)	q_8 (XOR)	q_{10} (OR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	0	1

1.3 Exemplos de execução

A Tabela 3 apresenta entradas testadas e o conteúdo final da fita 2.

Tabela 3 – Exemplos de entradas e resultados

Entrada (fita 1)	**Resultado (fita 2)**	**Situação**
`A110#101`	`100`	aceita
`X110#101`	`011`	aceita
`O110#101`	`111`	aceita
`OOO11#0101`	`0111`	aceita
`A1#0`	`0`	aceita

1.3.1 Rastreamento de `A110#101`

1. q_0: lê `A`, vai para q_1.

2. q_1: copia `110` para a fita 2; ao ler `#` vai para q_4.

3. q_4: recua a fita 2 até o início de `110`; vai para q_5.

4. q_5: calcula $1 \wedge 1 = 1$, $1 \wedge 0 = 0$, $0 \wedge 1 = 0$, deixando `100` na fita 2.

5. Lê $\square$ nas duas fitas e aceita em q_6.

1.4 Limitações

- Os operandos devem ter o mesmo comprimento. Se w_2 for mais longo, a fita 2 lê $\square$ durante o cálculo e a máquina para sem aceitar; se for mais curto, a máquina aceita deixando bits de w_1 não processados na fita 2.

- Não há validação de símbolos inesperados; nesses casos a máquina simplesmente para em estado não final.

2 CÓDIGO-FONTE

Para executar:

```
python3 main.py             # tabela-resumo dos exemplos
python3 main.py A110#101    # execução passo a passo
```

2.1 Módulo `fita.py`: representação das fitas

A classe `Fita` armazena os símbolos em um dicionário indexado pela posição, o que permite que a fita seja infinita nos dois sentidos. O atributo `cabeca` guarda explicitamente a posição da cabeça de leitura e escrita.

```
1  BRANCO = "□"
2  PASSO_DA_CABECA = {"R": +1, "L": -1, "S": 0}
3
4  class Fita:
5
6      def __init__(self, conteudo=""):
7          self.simbolos = dict(enumerate(conteudo))
8          self.cabeca = 0
9
10     def ler(self):
11         return self.simbolos.get(self.cabeca, BRANCO)
12
13     def escrever(self, simbolo):
14         self.simbolos[self.cabeca] = simbolo
15
16     def mover(self, direcao):
17         self.cabeca += PASSO_DA_CABECA[direcao]
18
19     def conteudo(self):
20         return "".join(s for _, s in sorted(self.simbolos.items()) if s != BRANCO)
21
22     def desenhar(self, inicio, fim):
23         celulas = []
24         for pos in range(inicio, fim + 1):
25             s = self.simbolos.get(pos, BRANCO)
26             celulas.append(f"[{s}]" if pos == self.cabeca else f" {s} ")
27         return "".join(celulas)
```

Código 2.1 – Módulo `fita.py`: representação das fitas

2.2 Importações e constantes

Definição do símbolo branco (□), do estado inicial e do conjunto de estados finais.

```
import sys
from collections import namedtuple
from fita import Fita
from printout import mostrar_passo_a_passo, mostrar_resumo
BRANCO = "□"
ESTADO_INICIAL = "q0"
ESTADOS_FINAIS = {"q6"}
```

Código 2.2 – Importações e constantes

2.3 Função de transição δ

Cada regra é uma `Transicao` indexada pela tupla (estado, símbolo lido na fita 1, símbolo lido na fita 2), correspondendo diretamente às transições do diagrama da Figura 1.

```
Transicao = namedtuple("Transicao", "proximo escreve1 escreve2 move1 move2")

DELTA = {
    ("q0", "A", BRANCO): Transicao("q1", "A", BRANCO, "R", "S"),
    ("q0", "X", BRANCO): Transicao("q2", "X", BRANCO, "R", "S"),
    ("q0", "0", BRANCO): Transicao("q3", "0", BRANCO, "R", "S"),

    ("q1", "0", BRANCO): Transicao("q1", "0", "0", "R", "R"),
    ("q1", "1", BRANCO): Transicao("q1", "1", "1", "R", "R"),
    ("q1", "#", BRANCO): Transicao("q4", "#", BRANCO, "R", "L"),

    ("q2", "0", BRANCO): Transicao("q2", "0", "0", "R", "R"),
    ("q2", "1", BRANCO): Transicao("q2", "1", "1", "R", "R"),
    ("q2", "#", BRANCO): Transicao("q7", "#", BRANCO, "R", "L"),

    ("q3", "0", BRANCO): Transicao("q3", "0", "0", "R", "R"),
    ("q3", "1", BRANCO): Transicao("q3", "1", "1", "R", "R"),
    ("q3", "#", BRANCO): Transicao("q9", "#", BRANCO, "R", "L"),

    ("q4", "0", "0"):    Transicao("q4", "0", "0", "S", "L"),
    ("q4", "0", "1"):    Transicao("q4", "0", "1", "S", "L"),
    ("q4", "1", "0"):    Transicao("q4", "1", "0", "S", "L"),
    ("q4", "1", "1"):    Transicao("q4", "1", "1", "S", "L"),
    ("q4", "0", BRANCO): Transicao("q5", "0", BRANCO, "S", "R"),
    ("q4", "1", BRANCO): Transicao("q5", "1", BRANCO, "S", "R"),

    ("q7", "0", "0"):    Transicao("q7", "0", "0", "S", "L"),
    ("q7", "0", "1"):    Transicao("q7", "0", "1", "S", "L"),
    ("q7", "1", "0"):    Transicao("q7", "1", "0", "S", "L"),
```

```
    ("q7", "1", "1"):    Transicao("q7", "1", "1", "S", "L"),
    ("q7", "0", BRANCO): Transicao("q8", "0", BRANCO, "S", "R"),
    ("q7", "1", BRANCO): Transicao("q8", "1", BRANCO, "S", "R"),

    ("q9", "0", "0"):    Transicao("q9", "0", "0", "S", "L"),
    ("q9", "0", "1"):    Transicao("q9", "0", "1", "S", "L"),
    ("q9", "1", "0"):    Transicao("q9", "1", "0", "S", "L"),
    ("q9", "1", "1"):    Transicao("q9", "1", "1", "S", "L"),
    ("q9", "0", BRANCO): Transicao("q10", "0", BRANCO, "S", "R"),
    ("q9", "1", BRANCO): Transicao("q10", "1", BRANCO, "S", "R"),

    ("q5", "0", "0"):         Transicao("q5", "0", "0", "R", "R"),
    ("q5", "0", "1"):         Transicao("q5", "0", "0", "R", "R"),
    ("q5", "1", "0"):         Transicao("q5", "1", "0", "R", "R"),
    ("q5", "1", "1"):         Transicao("q5", "1", "1", "R", "R"),
    ("q5", BRANCO, BRANCO):   Transicao("q6", BRANCO, BRANCO, "S", "S"),

    ("q8", "0", "0"):         Transicao("q8", "0", "0", "R", "R"),
    ("q8", "0", "1"):         Transicao("q8", "0", "1", "R", "R"),
    ("q8", "1", "0"):         Transicao("q8", "1", "1", "R", "R"),
    ("q8", "1", "1"):         Transicao("q8", "1", "0", "R", "R"),
    ("q8", BRANCO, BRANCO):   Transicao("q6", BRANCO, BRANCO, "S", "S"),

    ("q10", "0", "0"):        Transicao("q10", "0", "0", "R", "R"),
    ("q10", "0", "1"):        Transicao("q10", "0", "1", "R", "R"),
    ("q10", "1", "0"):        Transicao("q10", "1", "1", "R", "R"),
    ("q10", "1", "1"):        Transicao("q10", "1", "1", "R", "R"),
    ("q10", BRANCO, BRANCO):  Transicao("q6", BRANCO, BRANCO, "S", "S"),
}
```

Código 2.3 – Função de transição δ

2.4 Descrição das fases

Tabela usada apenas na exibição, associando cada estado à fase do algoritmo.

```
FASE = {
    "q0": "Fase 1: lê o operador",
    "q1": "Fase 2: copia w1 para a fita 2 (AND)",
    "q2": "Fase 2: copia w1 para a fita 2 (XOR)",
    "q3": "Fase 2: copia w1 para a fita 2 (OR)",
    "q4": "Fase 3: volta a cabeça da fita 2 ao início",
    "q7": "Fase 3: volta a cabeça da fita 2 ao início",
    "q9": "Fase 3: volta a cabeça da fita 2 ao início",
    "q5": "Fase 4: calcula w1 AND w2",
    "q8": "Fase 4: calcula w1 XOR w2",
    "q10": "Fase 4: calcula w1 OR w2",
    "q6": "Fim: aceita",
}
```

```
14 PASSO_DA_CABECA = {"R": +1, "L": -1, "S": 0}
```

Código 2.4 – Descrição das fases

2.5 Classe `MaquinaTuring`

A máquina mantém explicitamente as duas fitas, o estado atual e a função δ. O método `executar` aplica transições até alcançar um estado final ou não haver regra definida.

```
1  class MaquinaTuring:
2      def __init__(self, entrada, delta=DELTA):
3          self.fita1 = Fita(entrada)
4          self.fita2 = Fita()
5          self.estado = ESTADO_INICIAL
6          self.delta = delta
7          self.passos = 0
8
9      def transicao_atual(self):
10         return self.delta.get((self.estado, self.fita1.ler(), self.fita2.ler()))
11
12     def passo(self, regra):
13         self.fita1.escrever(regra.escreve1)
14         self.fita2.escrever(regra.escreve2)
15         self.fita1.mover(regra.move1)
16         self.fita2.mover(regra.move2)
17         self.estado = regra.proximo
18         self.passos += 1
19
20     def executar(self, ao_passar=None, limite=10_000):
21         while self.estado not in ESTADOS_FINAIS:
22             regra = self.transicao_atual()
23             if regra is None or self.passos >= limite:
24                 return False
25             lido = (self.fita1.ler(), self.fita2.ler())
26             anterior = self.estado
27             self.passo(regra)
28             if ao_passar:
29                 ao_passar(self, anterior, lido, regra)
30         return True
```

Código 2.5 – Classe `MaquinaTuring`

2.6 Exibição da execução

Funções responsáveis pela saída: o rastreamento passo a passo, com a troca de estado, a regra aplicada e o conteúdo das fitas, e a tabela-resumo.

```
def mostrar_passo_a_passo(entrada):
    mt = MaquinaTuring(entrada)
    inicio, fim = -1, len(entrada)
    largura = 3 * (fim - inicio + 1)

    def linha(n, transicao, regra, mt):
        f1 = mt.fita1.desenhar(inicio, fim)
        f2 = mt.fita2.desenhar(inicio, fim)
        print(f"{n:>5} | {transicao:<11} | {regra:<21} | {f1} | {f2}")

    def ao_passar(mt, anterior, lido, r):
        if FASE[mt.estado] != FASE[anterior]:
            print(f"      ⊢- {FASE[mt.estado]}")
        regra = f"({lido[0]},{lido[1]}) → ({r.escreve1},{r.escreve2},{r.move1},{r.move2})"
        linha(mt.passos, f"{anterior} → {mt.estado}", regra, mt)

    print(f"\nEntrada: {entrada}\n")
    print(f"{'passo':>5} | {'estado':<11} | {'lê → escreve, move':<21} | "
          f"{'fita 1':^{largura}} | {'fita 2':^{largura}}")
    print("-" * (48 + 2 * largura))
    print(f"      ⊢- {FASE[mt.estado]}")
    linha(0, mt.estado, "configuração inicial", mt)

    aceitou = mt.executar(ao_passar)

    print()
    if aceitou:
        print(f"ACEITA em {mt.passos} passos. Resultado (fita 2): {mt.fita2.conteudo()}")
    else:
        print(f"REJEITA: sem transição em {mt.estado} lendo "
              f"({mt.fita1.ler()}, {mt.fita2.ler()}).")

def mostrar_resumo(entradas):
    print(f"{'entrada':<12} {'resultado':<10} {'situação':<9} passos")
    print("-" * 40)
    for entrada in entradas:
        mt = MaquinaTuring(entrada)
        situacao = "aceita" if mt.executar() else "rejeita"
        print(f"{entrada:<12} {mt.fita2.conteudo():<10} {situacao:<9} {mt.passos}")
```

Código 2.6 – Exibição da execução

2.7 Ponto de entrada

Sem argumentos, executa os exemplos da Tabela 3; com argumentos, mostra o passo a passo de cada entrada.

```python
if __name__ == "__main__":
    if len(sys.argv) > 1:
        for entrada in sys.argv[1:]:
            mostrar_passo_a_passo(entrada)
    else:
        mostrar_resumo(["A110#110", "A110#101", "X110#101",
                        "0110#101", "00011#0101", "A1#0"])
        print("\nPasso a passo: python3 main.py A110#101")
```

Código 2.7 – Ponto de entrada

2.8 Exemplo de saída

A Figura 2 mostra a execução passo a passo da entrada `A110#101`. Cada linha indica a troca de estado, a regra de δ aplicada e o conteúdo das duas fitas, com a posição de cada cabeça entre colchetes. Ao final, a fita 2 contém `100`, resultado de $110 \wedge 101$.

```
Entrada: A110#101

passo | estado      | lê → escreve, move   |            fita 1            |            fita 2
      ├─ Fase 1: lê o operador
    0 | q0          | configuração inicial | □ [A] 1  1  0  #  1  0  1  □ | □ [□] □  □  □  □  □  □  □  □
      ├─ Fase 2: copia w1 para a fita 2 (AND)
    1 | q0 → q1     | (A,□) → (A,□,R,S)    | □  A [1] 1  0  #  1  0  1  □ | □ [□] □  □  □  □  □  □  □  □
    2 | q1 → q1     | (1,□) → (1,1,R,R)    | □  A  1 [1] 0  #  1  0  1  □ | □  1 [□] □  □  □  □  □  □  □
    3 | q1 → q1     | (1,□) → (1,1,R,R)    | □  A  1  1 [0] #  1  0  1  □ | □  1  1 [□] □  □  □  □  □  □
    4 | q1 → q1     | (0,□) → (0,0,R,R)    | □  A  1  1  0 [#] 1  0  1  □ | □  1  1  0 [□] □  □  □  □  □
      ├─ Fase 3: volta a cabeça da fita 2 ao início
    5 | q1 → q4     | (#,□) → (#,□,R,L)    | □  A  1  1  0  # [1] 0  1  □ | □  1  1 [0] □  □  □  □  □  □
    6 | q4 → q4     | (1,0) → (1,0,S,L)    | □  A  1  1  0  # [1] 0  1  □ | □  1 [1] 0  □  □  □  □  □  □
    7 | q4 → q4     | (1,1) → (1,1,S,L)    | □  A  1  1  0  # [1] 0  1  □ | □ [1] 1  0  □  □  □  □  □  □
    8 | q4 → q4     | (1,1) → (1,1,S,L)    | □  A  1  1  0  # [1] 0  1  □ | [□] 1  1  0  □  □  □  □  □  □
      ├─ Fase 4: calcula w1 AND w2
    9 | q4 → q5     | (1,□) → (1,□,S,R)    | □  A  1  1  0  # [1] 0  1  □ | □ [1] 1  0  □  □  □  □  □  □
   10 | q5 → q5     | (1,1) → (1,1,R,R)    | □  A  1  1  0  #  1 [0] 1  □ | □  1 [1] 0  □  □  □  □  □  □
   11 | q5 → q5     | (0,1) → (0,0,R,R)    | □  A  1  1  0  #  1  0 [1] □ | □  1  0 [0] □  □  □  □  □  □
   12 | q5 → q5     | (1,0) → (1,0,R,R)    | □  A  1  1  0  #  1  0  1 [□] | □  1  0  0 [□] □  □  □  □  □
      ├─ Fim: aceita
   13 | q5 → q6     | (□,□) → (□,□,S,S)    | □  A  1  1  0  #  1  0  1 [□] | □  1  0  0 [□] □  □  □  □  □

ACEITA em 13 passos. Resultado (fita 2): 100
```

Figura 2 – Saída do simulador para a entrada `A110#101`

3 CONCLUSÃO

A Maquina de Turing de duas fitas desenvolvida computa corretamente as operações AND, OR e XOR entre números binários de mesmo tamanho. O uso de uma segunda fita permitiu armazenar o primeiro operando e reaproveitá-lo como área de resultado, evitando idas e vindas na fita de entrada e mantendo o número de estados pequeno (11 estados). A estrutura em ramos paralelos também facilita a inclusão de novas operações, bastando adicionar um novo ramo com a tabela-verdade desejada.